UNIVERSITY of WASHINGTON

Software Design for Data Science

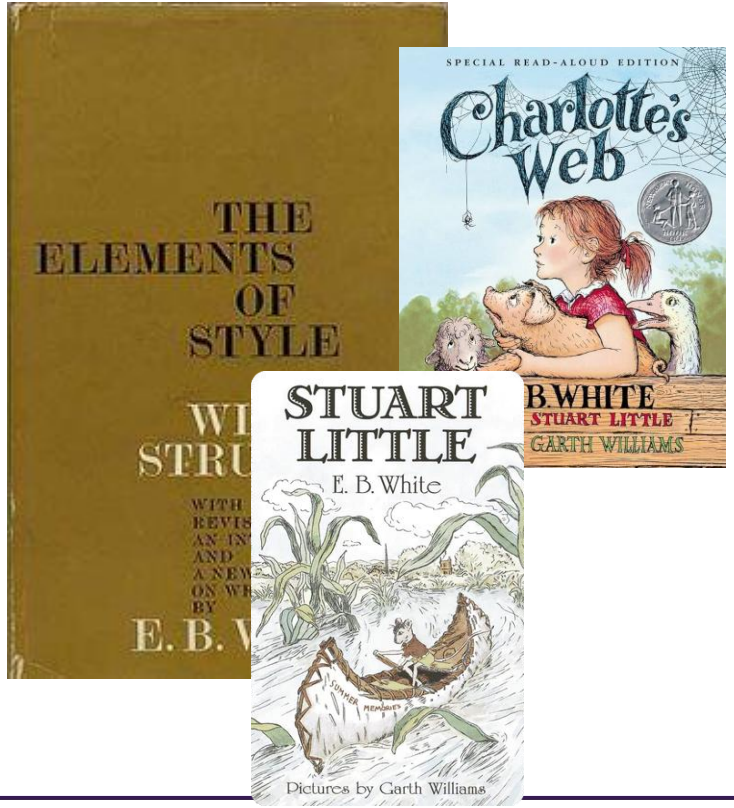
Style & Documentation

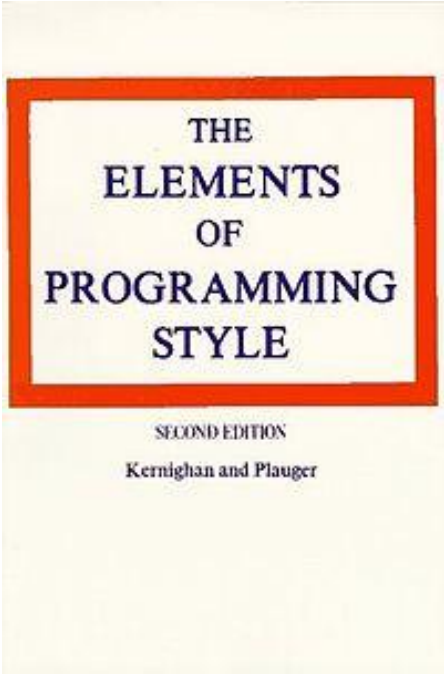
Naomi Alterman
University of Washington
February 24, 2026

Style and documentation…

BUT! Everyone loves animals

How to write vs how to write effectively





SURF-ing!

- Why is it important that people can read your code? (SURF)
  - Sustainable
    - New version of Python (4.X)
  - Understandable
    - Is it doing what you claim?
  - Reusable
    - Can it be incorporated into a larger project?
  - Fixable

Style

Ultimately, it doesn’t change how things work, just how approachable it is

Most important rule: consistency

Many different styles…

- PEP8 (most common)
  - “Python Enhancement Proposal #8”
- Google Python Style Guide
  - “More complete” PEP8

Use color! Use an editor!

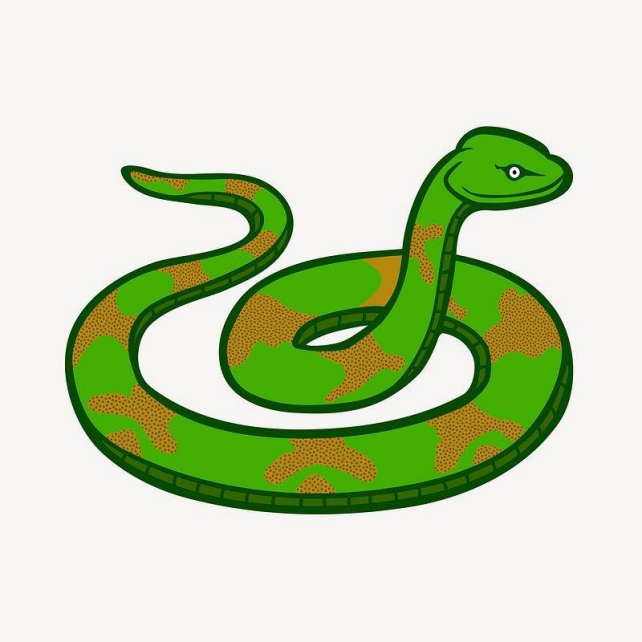
```
from statistics import mean
import numpy.random as nprnd
from statistics import stdev
def MyFuNcTiOn(ARGUMENT):
    m = mean(ARGUMENT)
    s = stdev(ARGUMENT)
    gt3sd = 0
    lt3sd = 0
    for m in ARGUMENT:
        if m > m + (s * 2):
            gt3sd += 1
        elif m < m - (s * 2):
            lt3sd += 1
    return(gt3sd, lt3sd)
def AnotherFunction(anumber = 1000, anothernumber = 1000):
    l = nprnd.randint(anothernumber, size = anumber)
    return(MyFuNcTiOn(l))
a,b=AnotherFunction(anumber = 1000, anothernumber = 1000)
print('found %d random values greather than 2 * sd and %d less than 2 * sd' % (a, b))
```

Tools for style checking

- Pylint
  - Extends from the *lint* tool for C

```
conda install pylint
pylint filename.py
pylint directory_name
```

- Also
  - `pycodestyle`
  - `flake8`



Exercise Part 1: Get Set Up

Install pylint

```
conda install pylint
```

Clone the demo repository (linked from the syllabus)

```
git clone
https://github.com/UWDATA515/style_documentation_example.git
```

Enter the repository directory

```
cd style_documentation_example
```

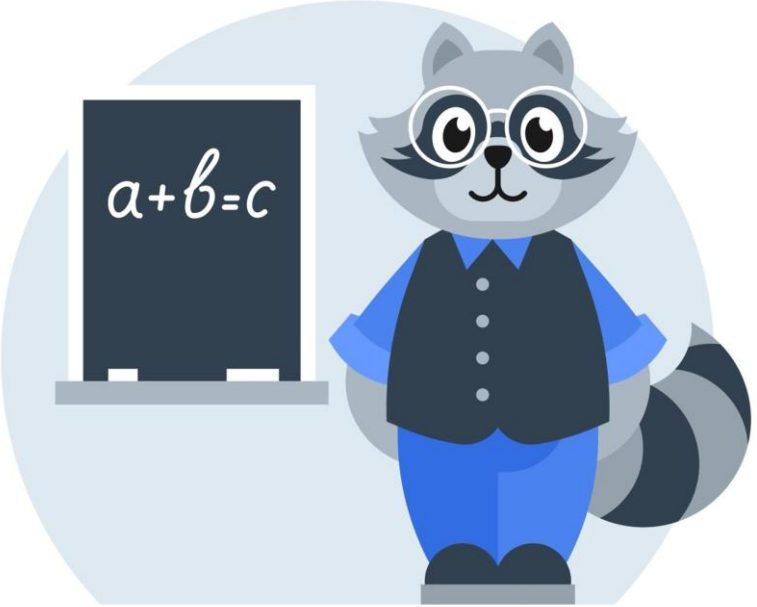
Run pylint

```
pylint style_demo.py
```

Don’t try to improve it yet!

The core pieces of style

- Spacing/indentation
- Line length
- Imports
- Code constructions
- Naming

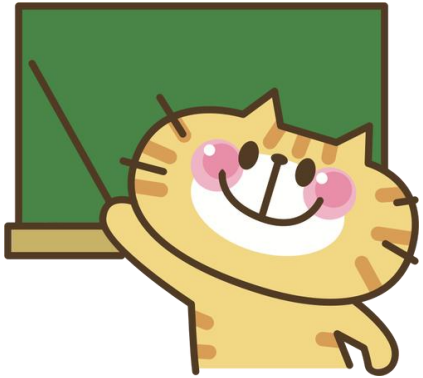


Made by FREE-VECTORS.NET

Spacing

- Indentation is so important in Python!
- Four spaces per indentation (not tabs!)
- Line wrapping - let’s see!

- Set your editor to convert tabs into spaces!



YES! indentation

```python
# Aligned with opening delimiter.
foo = long_function_name(var_one, var_two,
                         var_three, var_four)

# More indentation included to distinguish this from the rest.
def long_function_name(
        var_one, var_two, var_three,
        var_four):
    print(var_one)

# Hanging indents
foo = long_functio
    var_one, var_t
    var_three, var
```

```python
# Add a comment, which will provide some distinction in editors
# supporting syntax highlighting.
if (this_is_one_thing and
    that_is_another_thing):
    # Since both conditions are true, we can frobnicate.
    do_something()

# Add some extra indentation on the conditional continuation line.
if (this_is_one_thing
        and that_is_another_thing):
    do_something()
```

NO! Indentation

```
def long_function_name(
    var_one, var_two, var_three,
    var_four):
    print(var_one)
```

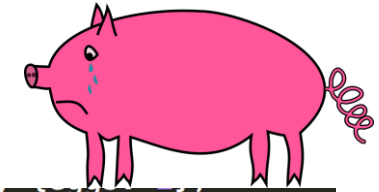
```
if (this_is_one_thing and
    that_is_another_thing):
    do_something()
```

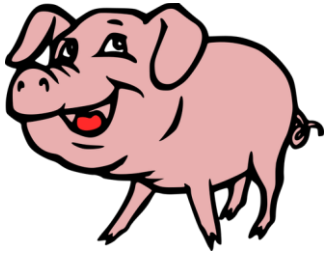
```
foo = long_function_name(var_one, var_two,
    var_three, var_four)
```

Line spacing

- Two blank lines around top-level functions
- Two blank lines around classes
- One blank line between functions in a class
- One blank line between logical groups in a function *(sparingly)*
- Extra blank lines between groups of groups of related functions *(why are they in the same file?)*

Whitespace





NO

```
spam( ham[ 1 ], { eggs: 2 } )
```

Brackets & braces

YES

```
spam(ham[1], {eggs: 2})
```

```
if x == 4 : print x , y ; x , y = y , x
```

: ; ,

```
if x == 4: print x, y; x, y = y, x
```

```
i=i+1
submitted +=1
x = x * 2 - 1
hypot2 = x * x + y * y
c = (a + b) * (a - b)
```

Mathy stuff

```
i = i + 1
submitted += 1
x = x*2 - 1
hypot2 = x*x + y*y
c = (a+b) * (a-b)
```

```
def complex(real, imag = 0.0):
    return magic(r = real, i = imag)
```

Function parameters

```
def complex(real, imag=0.0):
    return magic(r=real, i=imag)
```

- Trailing spaces at the end of a line

Line length

- Coding lines? Keep it to 79 characters
  - Most editors can show you the line position
- Comments & doc strings?
  - 72 characters
- Why? My monitor is big!
  - Open two files side by side? History?
  - Some teams choose to use a different max
  - Python core library is 79/72

Imports

At the top of the file!
(after any comments)

```
import os
import sys

import pandas as pd
import numpy as np

from phylodist.constants import TAXONOMY_HIERARCHY, PHYLODIST_HEADER
```

First: standard library imports

Blank line

Second: third party imports

Blank line

Last: local imports

Code constructions

Avoid compound statements!

```
if foo == 'blah': do_blah_thing()
else: do_non_blah_thing()

try: something()
finally: cleanup()

do_one(); do_two(); do_three(long, argument,
                             list, like, this)

if foo == 'blah': one(); two(); three()
```

Always include returns!

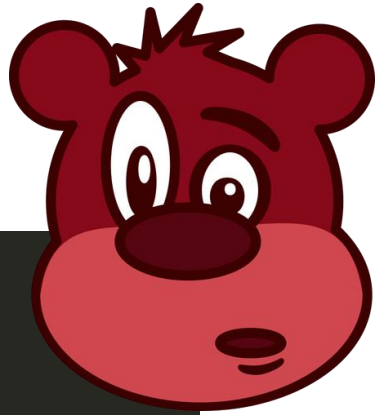
```
def foo(x):
    if x >= 0:
        return math.sqrt(x)

def bar(x):
    if x < 0:
        return
    return math.sqrt(x)
```

```
def foo(x):
    if x >= 0:
        return math.sqrt(x)
    else:
        return None

def bar(x):
    if x < 0:
        return None
    return math.sqrt(x)
```

Naming



- Choose variable names that won't be confused.

```
from statistics import mean
import numpy.random as nprnd
from statistics import stdev
def MyFuNcTiOn(ARGUMENT):
    m = mean(ARGUMENT)
    s = stdev(ARGUMENT)
    gt3sd = 0
    lt3sd = 0
    for m in ARGUMENT:
        if m > m + (s * 2):
            gt3sd += 1
        elif m < m - (s * 2):
            lt3sd += 1
    return(gt3sd,lt3sd)
def AnotherFunction(anumber, anothernumber):
    l = nprnd.randint(anothernumber, size = anumber)
    return(MyFuNcTiOn(l))
a,b=AnotherFunction(anumber = 1000, anothernumber = 1000)
print('found %d random values greather than 2 * sd and %d less than 2 * sd' % (a, b))
```

- How you name functions, classes, and variables can have a huge impact on readability

Naming

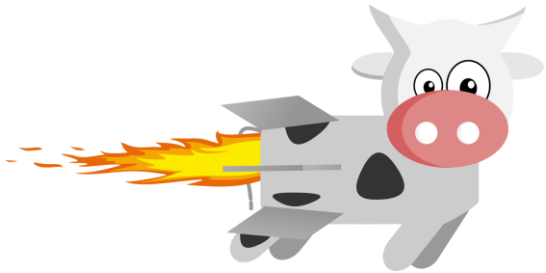
Avoid the following variable names:

- Lower case L (l)
- Upper case O (O)
- Upper case I (I)

These are unacceptable, terrible, and awful.
Why?

- Can be confused with 1 and 0 in some fonts
- Can be confused with each other (i.e. I and l)

Naming

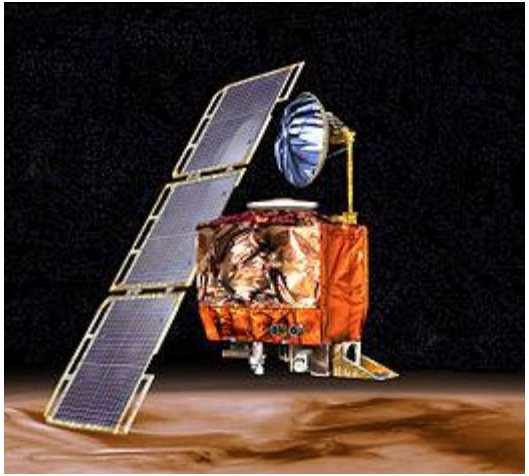


Functions

- Where units matter, append them…
  - Variable names also
  - Things go wrong…
    - Mars Climate Orbiter

… on September 23, 1999, communication with the spacecraft was lost as the spacecraft went into orbital insertion, due to ground-based computer software which produced output in non-SI units of pound-seconds (lbf×s) instead of the metric units of newton-seconds (N×s) <u>specified in the contract between NASA and Lockheed</u>.

- Wikipedia



655.2 million dollar mistake!

Naming

When naming functions and variables…

- Be consistent about pluralization / type Ids

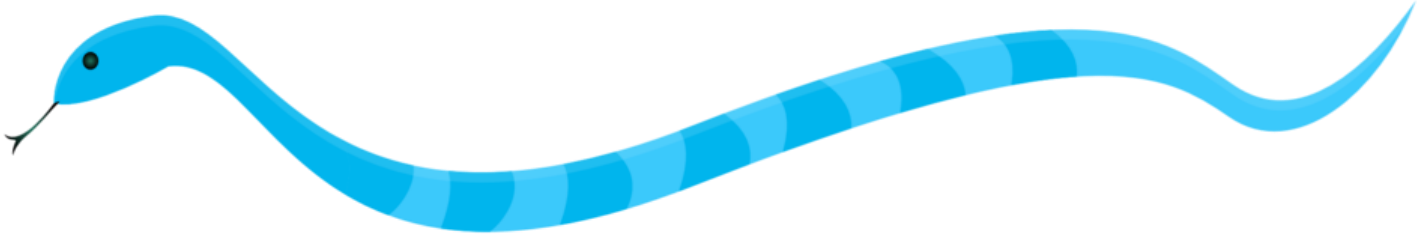
```
list_of_prime = [...]
primes = [...]
list_of_primes = [...]
prime_list = [...]
```

- Which do you prefer and why?
- Given your choice for the above, what would I name a variable that contained a collection of times? Users?

Naming

Conventions

- Module names should be short, lowercase
  - Underscores are OK if it helps with readability
- Package names should be short, lowercase
  - Underscores are frowned upon and people will speak disparagingly behind your back if you use them



Naming

Functions

- Some prefixes that can be used to clarify
  - compute
    Reserve for functions that compute something sophisticated, e.g. not average
  - get/set
  - find
  - is/has/can
    What kind of return value would you expect from a function with this kind of prefix?
  - Use complement names for complement functions
    - get/set, add/remove, first/last

Naming

Class names should be in CapWords

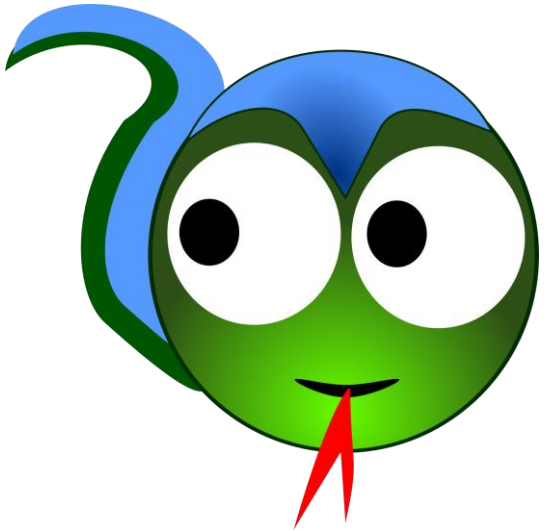
- SoNamedBecauseItUsesCapsForFirstLetterInEachWord
- Also known as CamelCase
- Notice no underscore!
- Much hate on the internet for Camel_Case

Naming

What naming convention should I use for exceptions?

WHY?

Naming



Functions

- Lowercase, with words separated by underscores as necessary to improve readability
- mixedCase is permitted if that is the prevailing style (some legacy pieces of Python used this style)
- Be consistent

Naming

- Naming conventions

Guidelines derived from Guido's Recommendations

Type	Public
Packages	
Modules	
Classes	
Exceptions	
Functions	
Global/Class Constants	
Global/Class Variables	
Instance Variables	
Method Names	
Function/Method Parameters	
Local Variables	

Exercise Part 2: Style

Run pylint on the style demo file:

```
pylint style_demo.py
```

Use an editor to get it to at least 8/10!

In addition, try to improve the following:

- Naming of functions and variables
- Pull common constants out into CONSTANTS

Ignore documentation-related issues for now.

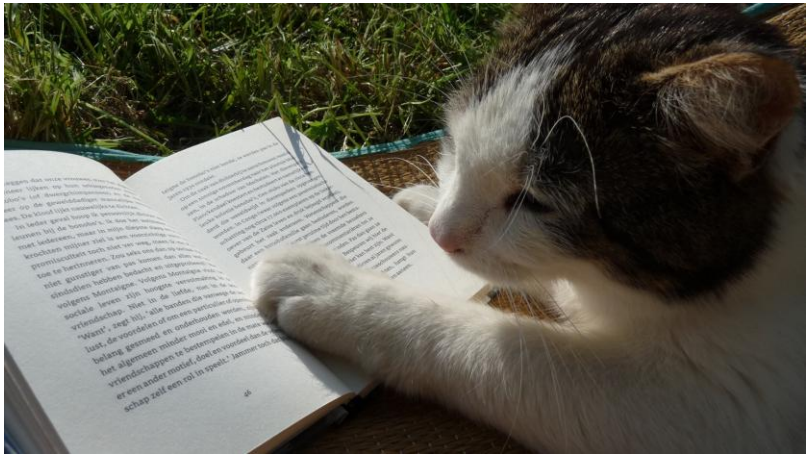
Most important rule: consistency

Documentation

Documentation

Documentation is for two types of people

Code comments



Code readers

README.md

Users

Documentation: README

- What kind of stuff goes in a repositories README.md?

https://github.com/kallisons/NOAH_LSM_Mussel_v2.0

Documentation: Comments

In Python:

```
# comment here
```

In shell scripting/bash:

```
# comment here
```

Always followed by a space before the rest of the comment.

Documentation: Comments

Some examples of bad comments (from the ‘net)

```
# For the brave souls who get this far: You are the chosen ones,
# the valiant knights of programming who toil away, without rest,
# fixing our most awful code. To you, true saviors, kings of men,
# I say this: never gonna give you up, never gonna let you down,
# never gonna run around and desert you. Never gonna m
# never gonna say goodbye. Never gonna tell a lie and
```

Don’t Rick Roll your readers!

```
# drunk, fix later
```

Uhm… Sigh.

Documentation: Comments

```
# Dear maintainer:
#
# Once you are done trying to 'optimize' this routine,
# and have realized what a terrible mistake that was,
# please increment the following counter as a warning
# to the next guy:
#
# total_hours_wasted_here = 42
```

Funny is funny, but don't troll. And what was the issue the writer encountered?

```
true = false
# Happy debugging suckers
```

At least it is logical?

http://stackoverflow.com/questions/184618/what-is-the-best-comment-in-source-code-you-have-ever-encountered

Documentation: Comments

Good comments…

- …make the code easy to read
- …balance terseness with clarity
- …discuss the function parameters and results

```
% parameters:
%   sequence = character string of nucleotide letters (ATCG)
% returns:
%   geneStarts = vector of start index into sequence of start codon
%   geneEnds = vector of stop index into sequence of stop codons
%                value is the first base of the stop codon
function [ geneStarts, geneEnds ] = callGenesFromSequence(sequence)
    ...
end
```

Documentation: Comments

Don’t comment bad code, rewrite it!

```
% this is so terrible
% I can't find the bug here but, just subtract the length of x from
%   the result and divide by the length
function meanX = averageX(x)
    meanX = sum(x) + length(x)
end
```

Then comment it

```
% parameters:
%   x = a vector of numerics
% returns:
%   meanX = the average of the vector x
function meanX = averageX(x)
    meanX = sum(x) / length(x)
end
```

Documentation: Inline Comments

- Comments on the same line as code

```
x = x + 1 # Increment x
```

- Use sparingly
  - Make sure it’s actually useful!
  - Make sure the line isn’t too long
  - Make sure the code is still readable

Documentation: Comments in Code

- Wrong comments are bugs

```
    # Unit test for the sweepFiles function to test bounds
    # checking on metadata parameters.
    def test_sweepFiles_metadataType(self):
        with self.assertRaises(TypeError):
            io.sweepFiles('examples', metadata=41)
```

- Don’t forget to update the comments!

Documentation: Comments in Code

- Don't insult the reader!

```
% compute the square root of the square of the distance
distance = sqrt(distanceSquared);
```

- If they're reading your code, they aren't that dumb

```
# Find the square of the 3D distance.
distance_squared = (x2-x1)^2 + (y2-y1)^2 + (z2-z1)^2
# Compute the square root of the square of the distance.
distance = math.sqrt(distance_squared);
# Make sure the distance is less than 3.5 Angstroms or error.
if distance < 3.5:
        # Throw an error.
        error('interatomic distance is less than 3.5 Angstroms')
else:
        # Add the distance to the list of distances.
        distances.append(distance)
```

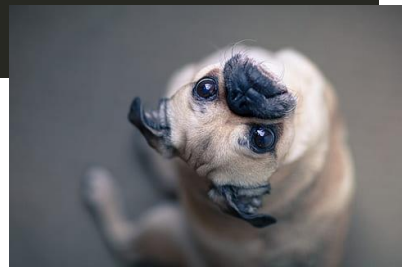
Corollary:

- Don't comment every line

Documentation: Comments in Code

Example: what's not great about this, other than too many comments?

```
# Find the square of the 3D distance.
distance_squared = (x2-x1)^2 + (y2-y1)^2 + (z2-z1)^2
# Compute the square root of the square of the distance.
distance = math.sqrt(distance_squared);
# Make sure the distance is less than 3.5 Angstroms or error.
if distance < 3.5:
        raise Exception('distance violation',
                        'interatomic distance is less than 3.5 Angstroms')
else:
        # Add the distance to the list of distances.
        distances.append(distance)
```



What if I want to change the cutoff distance? There are 3 places to change it!

Documentation: Comments in Code

Note: English!
Full sentences!

Better

```
# Find the square of the 3D distance and compute the sqrt,
#       then compare the distance to our cutoff (defined elsewhere)
#       and throw an error if the distance is too small
#       otherwise add the distance to our vector of distances.
distance_squared = (x2-x1)^2 + (y2-y1)^2 + (z2-z1)^2
distance = math.sqrt(distance_squared);
if distance < DISTANCE_CUTOFF:
        raise Exception('distance violation',
                        'interatomic distance is less than %.2f Angstroms'
                        % DISTANCE_CUTOFF)
else:
        distances.append(distance)
```

- Obvious constant
- Comments are consolidated and don't mention the cutoff

Documentation: Docstrings

When you provide a plain old string as the FIRST thing in a (function or class or module)

```
def sorted_list_difference(expected, actual):
    """Finds elements in only one or the other of two, sorted input lists.

    Returns a two-element tuple of lists.    The first list contains those
    elements in the "expected" list but not in the "actual" list, and the
    second contains those elements in the "actual" list but not in the
    "expected" list.    Duplicate elements in either input list are ignored.
    """
```

Use triple quotes

Documentation: Docstrings

What are they for?

- Documenting things users will need to know how to use
  - Functions
  - Modules
  - Classes

Documentation: Docstrings

Why?

- They can be processed by the *docutils* package into HTML, LaTeX, etc. for high quality code documentation (that makes you look smart).

Try it out when you've written some docstings:

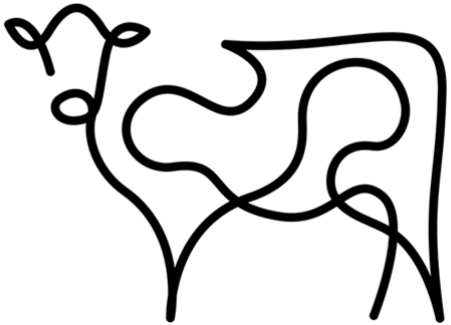
```
python -m pydoc my.module
```

(note: you leave the .py off your module name above!)

Documentation: Docstrings

One line for simple stuff

```
def three_way_cmp(x, y):
    """Return -1 if x < y, 0 if x == y and 1 if x > y"""
    return (x > y) - (x < y)
```



Documentation: Docstrings

Multiline for more complicated things

```
"""
A multiline doc string begins with a single line summary (<72 chars).

The summary line may be used by automatic indexing tools; it is
important that it fits on one line and is separated from the rest of
the docstring by a blank line.
"""

import sys
```

Documentation: Docstrings

For scripts intended to be called from the command line, the docstring at the top of the file should be a usage message for the script.

```
"""
Usage: my_program.py [-hso FILE] [--quiet | --verbose] [INPUT ...]

-h --help    show this
-s --sorted  sorted output
-o FILE      specify output file [default: ./test.txt]
--quiet      print less text
--verbose    print more text

"""
```

Documentation: Docstrings

For modules, list the classes, exceptions and functions (and any other objects) that are exported by the module, with a one-line summary of each.

https://github.com/numpy/numpy/blob/master/numpy/__init__.py

Documentation: Docstrings

For functions, include all of the good interface specification stuff we've talked about:

- Name
- Arguments
- Return values
- Side effects
- Edge cases
- Exceptions
- Examples

https://github.com/scikit-learn/scikit-learn/blob/main/sklearn/cluster/_dbscan.py

Exercise Part 3: Documentation

Run pylint on the style demo file:

```
pylint style_demo.py
```

Use an editor to get it to 10/10!

Lastly: Ignoring recommendations

- Sometimes you disagree with a PEP8 recommendation
- Or you made a very conscious choice to do it differently
- You can tell pylint to ignore the issue with a comment on the line before:

```
# pylint: disable=consider-using-f-string
```

- NOT OFTEN!
  - We WILL deduct points on your project if you use this without a good reason

Exercise 4: Adding Style to Continuous Integration

You *must* add linting to your continuous integration for your project:

https://github.com/UWDATA515/ci_example/blob/536d2f84c2a18dd3d82647cfb4d6dfd38ab52068/.github/workflows/build_test.yml#L95-L100

However, it probably won't pass the first time.

- Take this time to fix style and add documentation to your project
- Then add linting to your continuous integration GitHub Actions Workflow